Local Apex Runtime and Test Engine

Version 2.0 | 28 October 2026 | Greenfield architecture and delivery specification

Build the fastest practical path from an Apex code change to a trustworthy local test result. Execute real Apex and project metadata, explain failures precisely, and make compatibility boundaries visible. Speed comes from doing less repeated work and executing required work efficiently. Accuracy comes from explicit semantics and independent conformance evidence.

The implementation is a Rust compiler, evaluator and test engine with a native transactional record store. A TypeScript editor extension uses the same engine. The complete core, compatibility suite, benchmark harness and developer tooling will be open source under Apache-2.0. The working executable name is apx; final branding is a separate decision.

This specification defines the intended product, architecture, observable contracts and release gates. MUST identifies a mandatory requirement for the milestone that introduces a feature; SHOULD identifies a preferred design. Performance budgets are engineering targets, not measured results. Compatibility is earned feature by feature and profile by profile.

Contents

1. Product objective and competitive standard
2. Architectural decisions
3. Compatibility profiles and result integrity
4. System architecture and ownership
5. Source capture and metadata loading
6. Compiler and intermediate representation
7. Apex values and guest memory
8. Record store and transaction kernel
9. SOQL planning and execution
10. DML and automation lifecycle
11. Security and environment
12. Test lifecycle and asynchronous work
13. Limits cancellation and replay
14. Incrementality and parallel scheduling
15. CLI editor and debugging experience
16. Reports protocols and coverage
17. Conformance and accuracy engineering
18. Performance and competitive validation
19. Open source and distribution
20. Implementation sequence and release gates
21. Risks and architectural responses
22. Design checkpoint and reference policy

1 Product objective and competitive standard

OBJ-01 A developer MUST be able to open a supported Salesforce project, run an existing Apex test locally, inspect a failure, change the code and rerun it without deployment, an account, credentials or a network connection. The normal local loop must leave project source unchanged.

OBJ-02 Optimize time to a trustworthy result: elapsed time from command invocation or completed edit capture to a complete, policy-accepted result for the requested selection. Trust applies within the declared semantic profile, environment assumptions and assurance scope. It is not a claim that the project will deploy or behave identically under every Salesforce workload.

Advantage	Product commitment	Evidence required
Fast feedback	Native startup, incremental compilation, cheap test baselines, parallel execution	Cold, warm, edit-to-result and full-suite latency on a public corpus.
Accurate execution	Explicit Apex values, security, transactions and version policies	Independent platform observations, negative controls and regression cases.
Useful failures	Source assertions, record history, automation stack and query explanations	Diagnostics that locate the failing source and causal operation.

Advantage	Product commitment	Evidence required
Predictable operation	Offline defaults, bounded resources and reproducible replay	Egress tests, isolation tests, failure injection and replay equivalence.
Open development	Public semantics, fixtures, benchmarks and release provenance	Reproducible builds and contributions that need only Apex knowledge.

OBJ-03 Competitive performance MUST be reported alongside compatibility coverage. Compare latency on the common correctly supported workload and completion across the entire published workload. Blocked, omitted, cached or incorrectly executed tests do not count as execution throughput.

OBJ-04 The product boundary is a local evaluator and test engine. Production hosting, a complete Salesforce service emulator, Lightning rendering, org administration and deployment validation are separately scoped extensions. They must not delay the core test loop.

2 Architectural decisions

Decision	Chosen design	Reason and constraint
Engine	Stable Rust; pinned toolchain and dependencies	Control over representation, allocation and native distribution. Language choice alone proves no speed advantage.
Editor	TypeScript, VS Code Testing API and DAP	Familiar run/debug interfaces; semantics remain in Rust.
Front end	Parser adapter, owned syntax tree, binder, type checker, typed IR	Grammar parsing is not semantic validation.
Execution	Reference evaluator and compact register-bytecode VM	Establish behavior, then qualify lowering and optimized execution.
Guest memory	Per-test handle heap and tracing collection	Shared references, cycles and Apex identity independent of Rust ownership.
Storage	Typed rows, immutable baselines, per-test overlays, undo journals	Precise SOQL/DML semantics without full database cloning.
Queries	Reference scan evaluator and qualified indexed plans	Separate semantic rules from optimization.
Parallelism	Bounded worker processes, independent test contexts	Share immutable artifacts; contain mutable state and faults.
Incrementality	Content-addressed artifacts and conservative dependencies	Reuse work without hiding relevant tests.
License	Apache-2.0 for original code, fixtures and tooling	Permissive distribution with public contribution and provenance rules.

DEC-01 Core behavior MUST be explicit Rust contracts. Host overflow, string indexing, locale, hash iteration, SQL coercion and JSON number conversion must not accidentally determine Apex behavior.

DEC-02 Initial parsing SHOULD use tree-sitter-sfapex through its Rust bindings behind a replaceable adapter. Adoption requires the syntax acceptance gate. Improve or replace it if that gate fails; do not ship permissive parsing as compatibility. Tree-sitter's C implementation is an explicit FFI dependency, so the dependency graph is not entirely Rust. Tree-sitter bindings and Salesforce grammar.

DEC-03 Public source, documentation and established implementation patterns may inform the design. Reuse components with recorded revisions, licenses, attribution and conformance tests. The project owns its architecture, interfaces and semantic specification; no particular external runtime is required to build, run or validate it.

DEC-04 Defer JIT compilation, a general SQL engine, distributed storage, a native plugin ABI and a cloud control plane until measured needs justify them. SQLite may later store cache metadata or exported fixtures; its type system must not define Apex or SOQL behavior. SQLite types.

3 Compatibility profiles and result integrity

PRO-01 A semantic profile MUST identify platform release, allowed component API versions, org feature switches, security policy, standard schema, API signatures, limits, locale/time-zone data and conformance revision. Profiles have immutable content digests. Missing environment information must diagnose rather than invent a default org.

PRO-02 The first target is w27-core@1: a constrained Winter '27 profile with component API 68.0. Legacy component versions 65.0 through 67.0 are subsequent, separately qualified additions, including mixed-version call chains. Never rewrite component versions to make a project execute. Winter '27 is documented with API 68.0; that number does not imply coverage of every release feature. Winter '27 developer guide.

PRO-03 Platform release and source API version MUST remain separate inputs. Resolve rules through the relevant component, operation and execution frame, including user, access mode, namespace and entry point. Salesforce guidance changes default access behavior at API 67.0, making a universal security default unsound. Secure Apex Classes.

PRO-04 Capabilities MUST declare signature or metadata form, supported profiles, preconditions, effects, status, evidence IDs and exclusions. Status is qualified, modeled or unsupported. Qualified means that the declared contract passes its evidence suite, not formal proof or universal Salesforce equivalence.

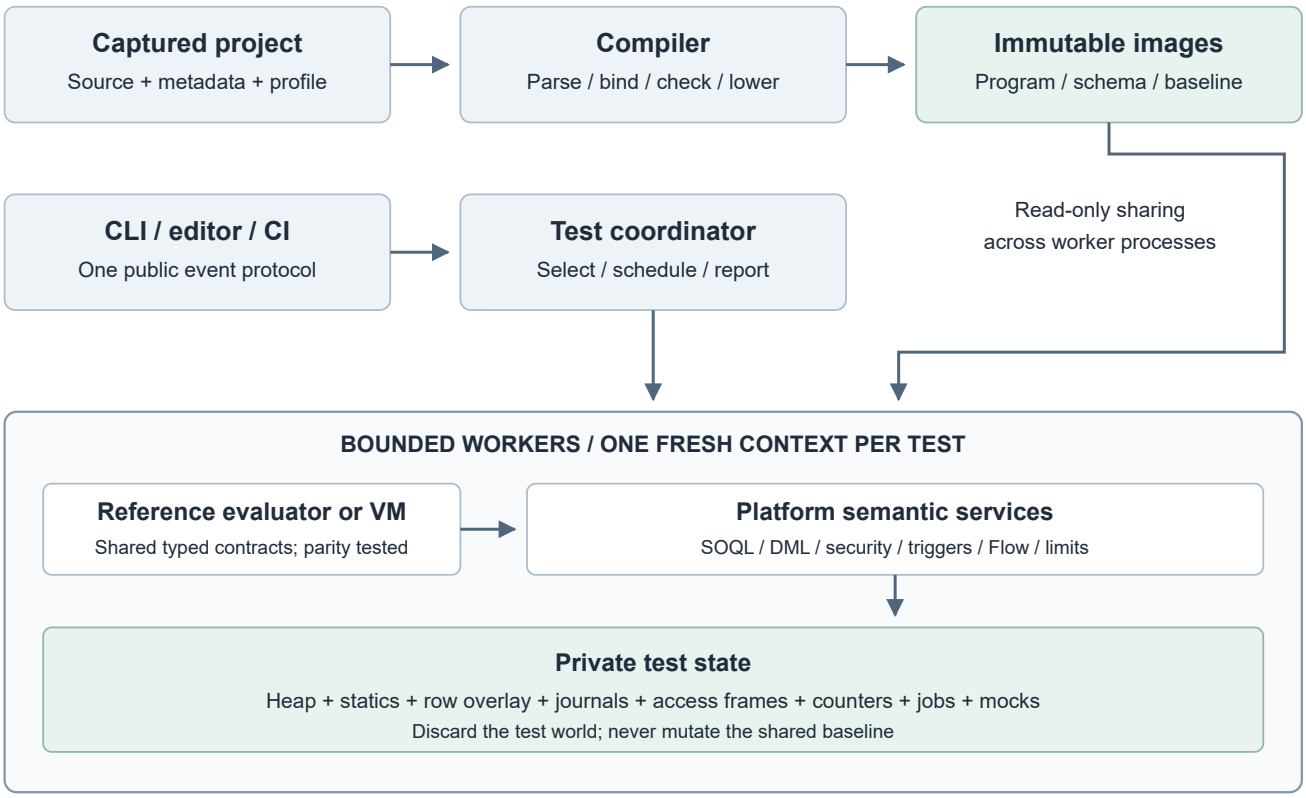
Dimension	Values	Meaning
Outcome	passed, failed, blocked, errored, cancelled, skipped	What happened to the selected test. A modeled run can still contain a real assertion failure.
Compatibility	qualified, modeled, unsupported	Whether executed semantics and required environment fit the published contract.
Selection	full, filtered, affected	What was requested and why. An affected run is not a full-suite result.
Assurance	functional, counted-limits; explicit resource exclusions	Properties evaluated. Cloud CPU, heap and service behavior are not inferred from local speed.
Policy acceptance	accepted, rejected	Whether the run satisfies CI policy, independently of raw execution outcome.

PRO-05 Default policy MUST reject modeled execution, unsupported dependencies, incomplete execution, failures, engine errors and cancellation. A preview policy may authorize named models for exploration, but results remain advisory and retain non-success status. Only complete qualified success within the declared assurance scope can be policy-accepted and return exit code 0.

PRO-06 A qualified functional pass MUST state resource and environment exclusions. A policy requiring Salesforce CPU or heap equivalence blocks when unavailable. Tests inspecting an unavailable resource counter block or explicitly use a named model. Useful functional testing must not imply certification of every platform limit.

PRO-07 Unsupported behavior MUST never return a convenience null, empty collection, zero or successful no-op. Reject statically known required unsupported operations before execution. For uncertain reachability, install runtime guards before effects. Active metadata that could influence a reached operation must be qualified or block it.

4 System architecture and ownership



The compiler produces immutable ProgramImage and SchemaImage artifacts. The runner combines these with a profile, fixture baseline and selection plan. Workers create fresh test contexts. CLI, debugger and editor all use the same language, query, DML and automation services.

Component	Owns	Must not own
apx-project	Source generations, packages, metadata, configuration	Execution state or credentials.
apx-compiler	Syntax, symbols, types, IR, lowering, source maps	User identity, mutable records or scheduling.
apx-values	Scalars, coercion, equality, hashing, object layout	Implicit host locale or database rules.
apx-runtime	Frames, heap, reference evaluator, VM, native calls	Project discovery or editor UI.

Component	Owns	Must not own
apx-schema / apx-store	Typed rows, constraints, baselines, overlays, journals	Guest heap pointers.
apx-query	SOQL binding, operators, planner, scan reference	DML lifecycle orchestration.
apx-platform	DML, triggers, formulas, Flow, security, limits, jobs	Alternate semantics for different clients.
apx-test	Discovery, setup, selection, workers, lifecycle	A second expression or query engine.
apx-protocol / apx-cli	Events, diagnostics, commands, DAP	Duplicated platform behavior.
apx-conformance	Fixtures, platform observations, equivalence	Automatic expectations copied from current engine output.

ARC-01 ProgramImage, SchemaImage, Profile and baselines MUST be immutable. TestContext exclusively owns statics, heap, stack, user/access frames, transaction overlays, limit frames, jobs, mocks, host observations, traces and diagnostics. Mutable Apex state must not escape into process globals or compiler caches.

ARC-02 Native methods MUST have typed, versioned signatures and explicit effects used by compiler and debugger alike. Calls root guest references, honor cancellation, charge logical operations and return structured results. Rust panics are engine faults, not Apex exceptions.

ARC-03 Execution exits MUST distinguish Return, ThrowApex, GovernorAbort, Unsupported, Cancelled and EngineFault. Only ThrowApex participates in ordinary Apex exception matching. Profiles define guest-visible unwinding; host cleanup always executes independently of guest catchability.

ARC-04 Package boundaries define ownership, not a demand to create every crate immediately. Begin with a small Cargo workspace. Public protocol types must not expose private memory layouts.

ARC-05 CLI, editor, daemon and workers MUST exchange versioned, length-bounded messages through child stdio or OS-local IPC. No TCP listener is enabled by default. Messages carry request IDs and captured generation/profile IDs. Apply backpressure and size limits, reject mismatches, and cancel specific requests. Non-child channels enforce OS user access controls.

5 Source capture and metadata loading

SRC-01 Support Salesforce DX package directories and classic Metadata API layouts. Resolve namespaces, dependencies and companion API versions before binding. Assemble decomposed metadata into logical objects. Missing parent files do not justify ignoring valid components; diagnose conflicts and missing required information at source locations.

SRC-02 Runs MUST use captured bytes, content hashes, logical paths, package identities and metadata from one immutable generation. Read and validate manifests twice, retrying if files change. Compilation and reports use captured bytes. Exported bytes are the reproducibility boundary; do not claim an atomic filesystem snapshot.

SRC-03 Configuration precedence is CLI arguments, project configuration, then documented defaults. Local commands MUST NOT inspect authentication stores, inherit arbitrary environment settings or execute project shell hooks. Only documented variables affect behavior; effective non-secret values appear in the manifest.

SRC-04 Inventory MUST distinguish schema information from behavior. Unknown inert UI attributes may be reported as ignored with reasons. Unknown active rules, formulas, relationships, permissions or automation cannot be silently ignored. Build an influence graph from operations to relevant metadata and dependencies.

SRC-05 Standard objects/APIs MUST come from versioned catalogues with provenance. Begin with Account, Contact, User and supported custom objects. Catalogue entries do not imply implemented behavior. Package signatures without bodies may support checking but must block execution unless a declared mock or implementation satisfies the contract.

SRC-06 doctor MUST inspect shape, compatibility and dependencies without executing Apex or reaching the network. check MUST distinguish invalid Apex from unsupported engine features. User exclusions are explicit in the selection report.

6 Compiler and intermediate representation

CMP-01 Parse Apex, embedded/dynamic SOQL and metadata expressions through distinct adapters. Preserve byte spans and spelling; convert deliberately to UTF-16 editor positions. Execution rejects missing/recovered/error nodes. Editor analysis may retain incomplete trees, separately from executable artifacts.

CMP-02 Binding MUST implement case/namespace rules, classes, interfaces, enums, properties, constructors, overloads, generic collections, inheritance, visibility and virtual dispatch. Do not resolve overloads by host casts or first match. Invalid source fails before unrelated side effects execute.

CMP-03 Typed IR MUST carry resolved symbols, explicit coercions, source locations, control flow and effects. Effects include exceptions, heap mutation, queries, DML, security, limits, time, randomness and jobs. Desugaring preserves evaluation order, short-circuiting, null behavior and exception locations.

CMP-04 Start with an unoptimized IR evaluator, then lower to compact register bytecode with explicit frames and resolved identifiers. Intern immutable names/constants and use dense symbol/field IDs. Optimizations preserve effects and logical accounting, including expressions whose values are discarded.

CMP-05 Bytecode MUST carry format version, compiler fingerprint, profile digest, constants, source map and integrity hash. Validate registers, branches, signatures, exception regions, native calls and operands before execution, including cached artifacts. Corrupt cache entries rebuild; malformed supplied artifacts are rejected.

CMP-06 Introduce constant evaluation, direct member slots, call-site caches, instruction fusion and query specialization separately. Each needs a disabled comparison mode. An optimization cannot change overflow, exceptions, observed limits or effect order. Defer JIT work until profiles demonstrate an interpreter bottleneck.

CMP-07 Parser qualification MUST cover accepted/rejected syntax, ambiguities, version gates and diagnostics. Independent parsers are useful cross-checks; platform compiler observations settle profile acceptance. Apex Dev Tools is an optional development cross-check, not a production JVM dependency. Apex parser project.

7 Apex values and guest memory

VAL-01 Value MUST represent null, Boolean, Integer, Long, Decimal, Double, String, Id, Date, Time, Datetime, Blob, enums, type tokens and guest references without loss. Use typed encodings where JSON numbers cannot preserve precision, identity or field state.

VAL-02 Decimal MUST use an exact coefficient and scale, arbitrary-precision fallback, and explicit profile rules for rounding, precision, overflow and conversion. Compact integers may optimize equivalent common cases. A fixed decimal library or metadata field scale is not automatically the complete arithmetic/storage contract.

VAL-03 Qualify String length/indexing, escaping, case, comparison and Unicode behavior. Date/time operations distinguish user zone, UTC instants and calendar dates with pinned time-zone data. Rust bytes, host local time and default collation are never implicit semantics.

VAL-04 Specify equality, ordering and hashing by type and context. Test mutable keys, user equals/hashCode, numeric coercion, SObject identity/content and 15/18-character IDs. Hashing must agree with equality; host map order is not a platform guarantee.

VAL-05 SObjects MUST distinguish presence, null value, projection membership, relationship loading and dirty state. Use separate masks or equivalent structure, not one null marker. Persisted rows hold canonical data; guest objects separately track fields loaded and assigned.

MEM-01 Use stable, generation-checked handles in a per-test heap. Preserve aliases/cycles and prevent stale-handle reuse. Roots include frames, statics, exceptions, native calls, mocks and jobs. Begin with tracing collection at safe points; discard the heap at test completion.

MEM-02 Collection must not change observable identity or values. Actual host memory and guest heap estimates are separate metrics. Rust ownership protects the implementation but does not automatically implement Apex's object model. Rust ownership.

MEM-03 Share immutable program and row baselines, not a captured live VM heap. Setup statics, objects, counters and jobs must not leak through whole-VM snapshots. Any future frozen guest heap optimization requires independent evidence for copied roots and reset rules.

8 Record store and transaction kernel

The store is a small in-process relational substrate for tests. It needs exact visible record behavior and cheap isolated test worlds, not durable multi-user transactions, network replication or a general SQL parser.

DB-01 SchemaImage MUST define object/field IDs, types, relationships, required/unique constraints, external IDs, supported formulas and object capabilities. Rows contain canonical values, never pointers or handles into a test's guest heap.

DB-02 A test view MUST combine an immutable baseline with its own inserted rows, replacements, tombstones and index deltas. Reads see its own writes and no other test's changes. Journals support savepoints. Bound overlay depth and compact at safe points without changing visible semantics.

DB-03 Initial indexes cover identifiers, relationship keys and declared unique/external-ID keys. Index and constraint state MUST roll back atomically with rows. Baseline indexes are read-only. Add range/composite indexes only with measured benefit and equivalence evidence.

DB-04 Distinguish fork_test_world(baseline), create_savepoint(transaction), rollback_to(savepoint), complete_transaction and discard_test_world. Database rollback does not imply restoring guest objects, statics, assigned IDs, counters or queued effects. The platform layer provides a transition policy for each state category.

DB-05 Query visibility, deletion/undelete, cursors, relationships and generated IDs MUST have profile contracts. Local IDs must be valid for supported operations and reproducible under replay; they do not imply a corresponding live record.

DB-06 Baseline serialization MUST be versioned and deterministic, including schema, row, fixture and profile digests. Cross-process sharing uses validated read-only images or mappings, not OS fork assumptions. Forking a test world and mutating a few rows must not copy the entire baseline.

DB-07 Bound row/index memory, spill files and temporary storage. Host exhaustion is an engine resource diagnostic, not a fabricated platform limit. Durable database storage is outside v1's disposable test-world contract.

9 SOQL planning and execution

QRY-01 Static and dynamic queries MUST use the same typed QueryIR. Parse dynamic strings against current namespace, schema and access context; bind values stay typed data. Guest values must never be concatenated into host SQL.

QRY-02 Begin with a simple scan reference using explicit projection, predicates, relationships, ordering and aggregation for supported forms. Indexed and specialized execution must match it and independent platform observations, including errors and logical limits.

QRY-03 Qualify null logic, string comparison, LIKE escaping, numeric coercion, date literals, relationships, duplicate handling, aggregates, LIMIT/OFFSET and ordering. Single-record assignment and collection materialization retain their distinct no-row and multiple-row behavior. Successful parsing does not qualify query execution.

QRY-04 Separate physical work from logical governor charges. An index cannot reduce a charge the platform contract requires. Moving security filters, projection checks or relationship expansion across operators must not change visible errors, values or accounting.

QRY-05 Unordered results remain semantically unordered. Stable local order may aid replay but is not a Salesforce order guarantee. An exploratory order-perturbation mode should expose accidental assumptions and label its results separately. Explicit ORDER BY remains authoritative.

QRY-06 Block unsupported locking, SOSL ranking, currency rules and special service queries. FOR UPDATE is not implemented by silently taking no lock. Later lock simulation needs contention fixtures distinct from ordinary isolated test execution.

QRY-07 explain MUST show logical query, plan, estimated/observed work, index choice, access policy and charges. Redact values by default. Optimized-versus-scan equivalence remains a permanent test mode.

10 DML and automation lifecycle

DML-01 All writes MUST pass through one coordinator for access checks, input preparation, constraints, trigger context, validation, automation, storage, result mapping and rollback. Native APIs and Flow must not bypass it.

DML-02 Model order of execution as a versioned state machine with phases, re-entry rules and effects. Publish the supported phase graph. Platform order and oracle traces determine it; a generic before/save/after pipeline is insufficient. Order of execution reference.

DML-03 Scalar/bulk insert, update, delete, undelete and upsert MUST define all-or-none behavior, partial results, error order, field writeback, external-ID resolution and nested effects. Partial success must model required retry/re-entry behavior, not merely catch one error per row. Qualify chunk boundaries and mixed outcomes.

DML-04 Test rollback independently for persisted rows, materialized SObjects, assigned IDs, statics, limits, savepoint validity and jobs. Whole-test reset is separate. Restoring all of TestContext on savepoint rollback would erase state that the program may retain and is forbidden as a blanket implementation.

DML-05 Triggers MUST receive qualified new/old lists/maps, flags, mutability and bulk contexts. Keep source frames through recursive DML. A chosen deterministic ordering must not be presented as a platform guarantee when the platform leaves order unspecified.

AUT-01 Formulas MUST compile to typed FormulaIR with explicit null, numeric, string, date and relationship semantics. Share values and services without assuming Apex and formula operators are identical. Diagnose cycles and unsupported functions.

AUT-02 Flow MUST compile to a typed graph with element identities, entry criteria, activation/version, order, variables, faults and transaction boundaries. Start with synchronous record-triggered/autolaunched forms. Apex/DML calls use shared services; traces and coverage map to original elements.

AUT-03 Compute the potential automation influence closure before mutation. Unsupported active Flow, workflow, duplicate rule, roll-up, assignment or relationship effects block affected operations. Unrelated unsupported metadata can remain only when its irrelevance is established; uncertainty broadens the closure.

AUT-04 Diagnostics MUST expose the causal chain from Apex operation through affected record and automation element to nested operation and final error. Provide bounded record-change history on demand. Useful explanation is a core feature, not an optional exhaustive logging exercise.

11 Security and environment

SEC-01 Model user identity, sharing, object permissions, field permissions, execution mode and namespace separately. Resolve explicit access clauses and version defaults at the relevant operation/frame. A full-access fixture user cannot stand in for all security scenarios.

SEC-02 The initial subset MUST include explicit users, supported profile/permission-set grants, object/field permissions, ownership, simple private/public sharing, runAs and supported user/system mode operations. Unsupported hierarchy, territory, restriction/scoping rules or package effects block affected cases.

SEC-03 Checks MUST cover relevant query predicates, relationships and writes, not just output fields. Describe, access checks, strip-inaccessible behavior and actual operations use consistent policy data while preserving each API's distinct errors/filtering.

SEC-04 Environment packs MUST be explicit, inspectable and versioned. They contain schema, users, grants, settings and synthetic data needed for a run. Missing requirements diagnose. Project-only analysis cannot certify parity with an unseen org configuration.

ENV-01 Local execution MUST have no network effects. HTTP/SOAP and supported external integrations use registered mocks; missing mocks fail or block under the API contract. No automatic cloud fallback, telemetry, license check, update request or upload occurs during tests.

ENV-02 An optional platform-oracle adapter MUST be a separate command/dependency boundary with explicit invocation and target environment. A blocked local test cannot invoke it. Remote operations get their own manifest; credentials stay out of fixtures, reports and replay bundles.

ENV-03 Workers receive only needed environment, inputs and output paths. Exclude native extension loading, arbitrary Apex filesystem access and project shell execution. Processes contain faults; stronger OS sandboxing is a separately tested capability, not implied by process separation.

12 Test lifecycle and asynchronous work

TST-01 Discover tests from captured source and supported annotations. Validate signatures and selection before execution. A selection plan records stable identities, generation, exclusions and reasons. Empty selection is an error unless allow-empty is explicit.

TST-02 Setup builds the qualified database baseline for a class. Construct each test context using an explicit setup-to-test transfer policy. Reinitialize static/heap state; qualify initial counters and other platform state rather than assuming every category starts at zero. Do not carry a setup VM forward. Within-run reuse requires state-transfer fixtures; cross-run setup-result caching is deferred.

TST-03 startTest/stopTest MUST use an explicit stack of governor and async contexts, not reset the whole test. Qualify invalid calls, restoration of counters, savepoint effects, eligible work and drained-job errors per profile.

TST-04 Every selected test MUST reach one terminal outcome unless the host is forcibly terminated. Setup is a first-class result: an Apex setup failure makes the run failed, unsupported setup rejects compatibility, and engine faults remain errored. Dependent blocked tests retain that cause without changing its category. Failures, crashes, cancellation and host budget breaches cannot leak state.

TST-05 SeeAllData and metadata-visible test behavior need separate contracts. Local fixture visibility is not real org data access. Unspecified required org state blocks. Integration tests that commit externally or call live services are outside the local unit-test profile.

ASY-01 Distinguish future, Queueable, batch, scheduled and event-driven jobs. Each family needs payload serialization, transaction boundaries, user/access context, limits and lifecycle evidence. Running every callback immediately in the caller's frame is not a generic async implementation.

ASY-02 stopTest drains only work eligible under the profile's chaining rules. Do not recursively drain all descendants merely to finish. Surface async failures and unsupported continuations. Deterministic scheduling supports replay, not a claim of production interleaving or throughput.

TST-06 Baseline reuse, ordering and worker count MUST preserve semantic outcomes. Compare serial, shuffled, parallel and reuse-after-failure runs. Retain every retry attempt. Retries are diagnostic: a later pass cannot erase an earlier failed, errored or cancelled attempt or make the aggregate policy-accepted.

13 Limits cancellation and replay

LIM-01 Charge counted limits at semantic operations with profile rules. Include statements/rows, recursion, jobs, callouts and introduced capabilities, including failed/rolled-back operations where required. VM instruction or query-plan changes must not change these counters.

LIM-02 Salesforce CPU/heap are not host wall time, instruction count or Rust resident memory. Functional profiles MUST declare exclusions. Optional estimators have named, versioned models and distinct fields; resource APIs cannot silently expose host values as platform measurements.

LIM-03 Governor termination is distinct from catchable guest exceptions. Fixtures cover budget minus one, budget, budget plus one, nested contexts and cleanup. Salesforce describes governor exceptions as unhandleable; boundaries and state effects still require profile evidence. Apex governor limits.

RUN-01 All long operations MUST observe cancellation and budgets, including native calls, scans, parsers and report serialization. Poll the VM at bounded safe points; enforce worker deadlines against infinite loops. Stopped tests remain cancelled or errored.

RUN-02 Time, randomness and identifiers MUST use a host-observation interface. Functional execution records necessary nondeterministic inputs without forcing frozen time or deterministic cryptographic output. Replay supplies recorded inputs and detects mismatched consumption. Fixed-clock/altered-randomness simulation is an explicit mode with separate assurance.

RUN-03 Replay exports MUST include exact source/metadata, engine/profile IDs, fixtures, selection, configuration and required observations. Export is explicit and local with an inspectable manifest. Redacted or missing inputs make incomplete replay visible; no automatic upload.

RUN-04 Use bounded trace buffers with causal events and source identities. Value detail is configurable and sensitive fields redact by default. Report dropped events. Lightweight diagnostics and complete replay recording are distinct modes with separately measured overhead.

14 Incrementality and parallel scheduling

INC-01 Cache syntax, bound interfaces, checked bodies, bytecode, schema and plans by actual dependencies. Keys include profile, compiler/bytecode ABI, packages, metadata, catalogue and options. Do not cache mutable statics, cross-user access decisions, live test state or test verdicts in v1.

INC-02 Distinguish body, public-interface, schema and profile changes. A body edit may reuse unaffected interfaces; a trigger or grant can affect distant tests. Handle cycles and namespaces deterministically. Cache corruption falls back to a correct cold build.

INC-03 Affected-test selection MUST be conservative and explainable. Observed coverage alone cannot prove irrelevance. Reflection, dynamic queries, configuration and uncertain automation dependencies widen selection to a package or full suite. Coverage can improve prioritization, not erase unknown dependencies.

INC-04 Watch captures new generations after edits and cancels or labels superseded work. Late old-generation results cannot replace current editor state. Audit mode compares affected selection with complete execution.

INC-05 --changed compares against an explicitly recorded complete baseline generation for that workspace and profile. If that baseline or dependency evidence is missing, run the complete requested suite. Bound cache disk space and retained generations; evict only artifacts without active readers. Do not accumulate immutable generations indefinitely in watch mode.

PAR-01 Share read-only artifacts across workers, with independent mutable contexts. Default concurrency respects CPUs and a memory budget; expose --jobs. Begin with one active test per worker. Parallel tests do not imply concurrent transactions within one test.

PAR-02 Persistent worker reuse requires fresh-context evidence. Recycle workers after faults/resource thresholds. Worker death marks the active test errored and accounts for unstarted assignments; reruns are explicit attempts.

PAR-03 Scheduling may favor recent failures or short tests while preserving the selected set. Report that policy. Final results sort by stable identity with attempt IDs, independent of event arrival. Merge coverage/counts exactly once.

15 CLI editor and debugging experience

The CLI is the stable integration surface. Provide useful diagnostics quickly without requiring a separate dashboard. The editor uses native discovery, run/debug controls, failure locations and coverage decoration. VS Code Testing API.

```
apx doctor --project .
apx check --project .
apx test --filter AccountServiceTest.*
apx test --changed --watch
apx test --jobs 4 --jsonl results.jsonl --junit results.xml
apx test --coverage coverage.json --record run.apxrun
apx exec --file scripts/example.apex
apx explain --query-file queries/accounts.soql
apx capabilities --profile w27-core@1 --json
apx replay run.apxrun
apx debug --test AccountServiceTest.createsAccount
```

UX-01 Failures MUST show expected/actual values, original Apex location and a useful stack. Database errors add object/field and cause; unsupported results add capability ID, dependency path and next action. Preserve context without dumping secrets or every record.

UX-02 CLI/editor/CI MUST present selection, outcome, compatibility and assurance consistently. Advisory results cannot become ordinary green ticks. Show exact unsupported counts and tests, including those blocked before execution.

UX-03 DAP MUST support launch, breakpoints, continue/pause, step-in/over/out, stacks, scopes, variables and exception stops for qualified forms. Map to captured Apex/Flow source and expose generation mismatches. Advertise only implemented capabilities. Debug Adapter Protocol.

UX-04 Debug evaluation MUST use a restricted side-effect-free subset. Methods, property getters and toString overrides are not presumed pure. Evaluation cannot change records, statics, counters or jobs. Pause suspends the host wall deadline while retaining cancellation and memory bounds.

UX-05 The extension MUST use the same native engine and protocol as CLI. It renders/manages sessions without implementing Apex evaluation, queries or alternate policies. Protocol mismatch produces a clear error.

UX-06 exec runs captured anonymous Apex in a fresh disposable world using the selected profile/environment and its entry-point access rules. Transactions/jobs follow their qualified contracts; completion discards the world unless supported export is explicitly requested. Apply the same compatibility, assurance, cancellation and exit policy. Fixture inputs are not persistently modified.

16 Reports protocols and coverage

REP-01 Version public events separately from engine and profile. JSONL is one UTF-8 object per line; human progress goes to stderr when stdout carries events. Every event has schemaVersion, runId, sequence and generationId. Journal runs and atomically finalize reports.

REP-02 Emit run_started, discovery_completed, setup_started/setup_finished where applicable, test_attempt_started/test_attempt_finished for each attempt, one aggregate test_finished per selected test, and run_finished. Test aggregates reference every attempt; blocked tests can have none. Summaries retain setup causes, completeness, counts, scope, assurance and exit code. A killed process leaves an incomplete journal, not an assumed terminal result.

```
{
  "schemaVersion": 1,
  "event": "test_finished",
  "runId": "example-run",
  "sequence": 17,
  "generationId": "example-generation",
  "testId": "AccountServiceTest.createsAccount",
  "attemptIds": ["example-run:test-1:attempt-1"],
  "outcome": "passed",
  "compatibility": "qualified",
  "policyAcceptance": "accepted",
  "profile": "w27-core@1",
  "assurance": ["functional", "counted-limits"],
  "excludedAssurance": ["salesforce-cpu", "salesforce-heap"],
  "modelsUsed": [],
  "durationNs": 4200000
}
```

Example identifiers are illustrative. Production manifests contain SHA-256 content IDs for source, profile, metadata, fixtures, configuration and catalogue, plus engine build and host platform. Test events reference that immutable manifest rather than repeating it.

REP-03 Diagnostics MUST include code, phase, severity, message, source range, affected test/operation, relevant capability, causal IDs and remediation. Distinguish invalid source, missing environment, unsupported semantics, assertions, governor termination and engine defects.

REP-04 Exit codes are 0 for complete qualified policy-accepted success, 1 for test or setup Apex failures, 2 for configuration/parse/type/selection errors, 3 for engine/host failures, 4 for modeled execution or compatibility/assurance rejection, and 130 for cancellation. Mixed-result precedence is 130, 3, 2, 4, 1, 0; preserve all categories. Preview execution never converts an advisory result to code 0.

REP-05 JUnit MUST represent unsupported, cancelled and host-incomplete tests as errors with causes. Assertion failures use failures. Advisory passes use skipped entries with named model reasons; user-requested skips remain distinct in the companion manifest. Represent setup failures as dedicated suite errors/failures. Unselected exclusions remain in the manifest. Process exit remains authoritative.

REP-06 Define discovered, selected, excluded, started and terminal counts separately. In a complete run, selected equals the sum of terminal outcome counts; count each test once regardless of attempts. Started means at least one executed attempt and overlaps terminal categories. Exclusions are outside selection with reasons. Unsupported tests cannot be auto-excluded; selected skips require an explicit user action and do not satisfy complete qualified success.

COV-01 Maintain source-mapped executable sites and per-test hit sets. Distinguish line, branch and Flow-element coverage. Exclude test/generated engine code from default application coverage. Zero executable lines is not applicable, not 100 percent.

COV-02 Offer platform-qualified coverage only when attribution and setup rules pass differential fixtures. A setup-inclusive developer view must have a distinct name. Coverage carries file digests/generation; stale results cannot annotate edited source as current.

17 Conformance and accuracy engineering

Accuracy is a maintained body of evidence. Agreement between two local evaluators is useful but cannot independently certify shared platform semantics. A suite must exercise failures, boundaries and state transitions, not merely successful examples.

QA-01 Maintain three comparisons: IR evaluator versus bytecode VM; scan queries versus optimized plans; local observations versus Salesforce observations. Shared value/platform libraries limit independence of the first two, so platform evidence remains necessary.

QA-02 Fixtures MUST contain minimal source, metadata/environment, profile, inputs, observation schema, expected behavior and provenance. Record release, settings, features, modes and component versions with oracle observations. Pin documentation revisions or observation dates for version-sensitive rules.

QA-03 Compare outcomes, typed values, exception categories, source attribution, record graphs, visibility, side effects and limits. Normalize IDs with relationship-preserving aliases and times/order only when permitted. Never normalize away precision, denied access, omitted effects or unexpected failures.

Family	Discriminating cases
Front end	Valid/invalid syntax, overloads, namespaces, visibility, versions, source ranges.
Values and heap	Decimal boundaries, Unicode, time zones, aliases, cycles, mutable keys, equality.
Queries	Nulls, binds, projection errors, relationships, aggregates, security, scan/index equivalence.
DML	Bulk boundaries, partial success, external IDs, re-entry, savepoints, rollback state.
Automation	Validation order, formula errors, Flow faults, activation, influence closure.
Tests	Setup visibility, statics, start/stop contexts, negative controls, async eligibility.
Security/versions	Modes, runAs, defaults, denied fields, mixed-version call chains.
Reliability	Cancellation, memory boundaries, worker death, corrupt cache, incomplete reports.
Reuse	Cold/warm equality, source/metadata edits and deletions, order and worker counts.
Unsupported	Unknown APIs/schema, active unsupported metadata, missing mocks, unavailable assurance.

QA-04 Promoted capabilities MUST have positive, negative and boundary evidence. Add property tests and fuzzing for parsers, types, values, bytecode, queries and metadata. Minimize each discovered mismatch into a regression fixture before accepting its fix.

QA-05 Deliberate engine mutations MUST corrupt arithmetic, bypass access checks, skip validation/triggers/limits, leak baseline data and return defaults for unknown APIs. The suite must detect each selected mutation. Golden outputs copied solely from the implementation are not independent evidence.

QA-06 CI runs offline fixtures on supported hosts, with scheduled oracle qualification for maintainers. Contributors can run saved observations without Salesforce access. Updating remote expectations requires review, not automatic snapshot refresh after failures.

QA-07 Qualification MUST compare repeated shuffled runs and worker counts 1, 2 and default. Compare semantics, not timing/event arrival. Known mismatches are fixed or explicitly removed from qualified scope and reported, never hidden behind an untracked allowlist.

QA-08 Freeze five representative project corpora before v1: language-heavy, data/trigger-heavy, automation/security-heavy, namespace/package-heavy and intentionally unsupported. Publish requested/qualified counts for all. At least three must be redistributable; the remainder need reproducible access or documented public synthetic counterparts.

18 Performance and competitive validation

PER-01 Freeze inputs and an exact machine at P0: native release builds, 8 logical CPUs, 16 GB RAM and local SSD as the planning envelope. Record CPU model, OS, power mode, build flags and hashes. Budgets below are proposed targets to validate during the prototype, not achieved results.

Workload	Target	Included work
Small cold check	p95 at most 1 second	New process, empty engine cache, 100 units / 10k source lines / 20 objects.
Small cold focused test	p95 at most 1.5 seconds	Capture, compile, setup, one data/trigger test and final report.
Warm focused test	p95 at most 100 ms	Resident program, fresh context, 20 writes, 10 queries and a bulk trigger.
Body edit to result	p95 at most 250 ms	Capture, invalidation, recheck, one affected data test; debounce reported separately.
Mixed 1000-test suite	p95 at most 15 seconds with one worker; 5 seconds with eight	Warm compilation, fresh setup/execution, at least 300 data/automation tests, 50k baseline rows.
Large cold check	p95 at most 3 seconds	1000 units / 100k lines / 100 objects, empty engine cache.
Memory	At most 1 GiB aggregate resident memory	Four-worker mixed suite; coordinator/children included, shared-page methodology disclosed.
Cancellation	Acknowledge within 100 ms; workers stopped within 1 second	Long loops, queries and native calls under normal host load.

PER-02 Distinguish new-process/empty-cache, new-process/populated-cache and resident-daemon runs. State OS page-cache conditions. Include capture, parse, bind, schema, setup, execution, reporting and IPC; microbenchmarks do not replace the workflow.

PER-03 For p95 acceptance, collect at least 100 measured runs per workload/configuration, with warmups separate. Publish quantile estimators, raw samples and confidence intervals. Smaller samples can report medians/ranges but cannot establish p95 gates. Randomize tool order and disclose variance. Investigate credible regressions over 10 percent on stable benchmarks before acceptance.

PER-04 Competitive stretch target: at least 2x lower median edit-to-qualified-result latency than the fastest compared local alternative on three common-surface project workloads, without weaker semantics or reduced selected work. Publish common-surface latency and full-corpus completion. Missing the target must drive honest bottleneck analysis, not altered denominators.

PER-05 Cloud comparisons separate deployment, queueing, execution and retrieval. Developer latency is valuable but network/queue delay is not interpreter speed. Public speed multipliers require tool versions, configurations, raw samples and matched-result checks.

PER-06 Optimize measured costs: repeated parse/bind work, baseline copying, query access, hot allocations, scheduling, then VM specialization. Every optimization must pass semantic, isolation, debug-source and accounting equivalence before its speed improvement counts.

19 Open source and distribution

OSS-01 Publish original core, CLI, extension, specs, profiles, harness, public fixtures and benchmarks under Apache-2.0. Preserve third-party licenses/notices. The license includes contribution and patent terms; distribute its actual text. Apache License 2.0.

OSS-02 Use DCO sign-off and a clear contributor guide. Identify reused source/licenses. Apex-only fixture contributions are first-class: a minimal case and observed expectation should be enough to start resolving a gap. Developer Certificate of Origin.

OSS-03 Document repository locations for crates, editor, specifications, profiles, fixtures, benchmarks and release tooling. Publish machine-readable capability gaps. Generated source includes generator version and reproduction commands.

OSS-04 Assign maintainers for compiler/runtime, data/automation, conformance and tooling. Semantic changes need regression evidence; optimizations need benchmarks; public contracts/profile changes need reviewed design notes. Community proposals must not depend on private roadmap access.

OSS-05 Version engine, event protocol, profiles and cache formats independently. Semantic fixes can invalidate artifacts without changing CLI compatibility. Profile digests are immutable; replacements get revisions and migration notes. Document semantic-versioning rules for public interfaces. Semantic Versioning.

OSS-06 Before v1, publish contribution, security, maintainer/governance and conduct policies, including a private vulnerability-reporting route. Provide an Apex fixture walkthrough that requires neither Rust experience nor Salesforce credentials. The public issue process must distinguish engine defects from declared capability requests.

REL-01 Ship Windows x64, Linux x64 and macOS arm64, adding platforms after qualification. Pin Rust and lockfiles. Users must not need Rust, a JVM, Docker or a database server for normal tests. The editor may use its host JavaScript runtime.

REL-02 Releases MUST include checksums, SBOM, notices, catalogue, conformance summary, benchmarks and provenance. Test installation, offline startup, a positive case and deliberate failure on each host. Document offline verification as well as online attestation.

REL-03 Updates/fetches are explicit commands. Treat source, metadata and archives as untrusted: bound parsing, reject traversal, disable XML external-entity fetching, validate caches and constrain report destinations to explicit paths. No background requests occur during tests.

REL-04 No account or hosted license is required for core capabilities. Support, integrations or hosted services may fund development but cannot make local correctness depend on a private service.

REL-05 Independently rebuild each supported target from pinned source/tool inputs and compare unsigned payloads byte for byte. Handle signing/notarization envelopes separately with documented verification. Build provenance alone is not reproducibility. An unexplained payload mismatch fails the reproducible-build release gate.

20 Implementation sequence and release gates

Deliver an executable vertical slice before expensive optimization. Phases describe intended scope; broad platform coverage is not promised within an arbitrary schedule.

Phase	Deliverable	Completion gate
P0 Architecture proof	Fixture/profile contracts; parser; values; IR evaluator; basic store; vertical slice	Compile a class, Decimal arithmetic, insert/query, trigger, two isolated tests and deliberate failure. Oracle comparison and initial benchmarks.
P1 Useful alpha	VM; core APIs; basic SOQL/DML; triggers/validation; setup; simple security; CLI/reports	At least 500 meaningful fixtures; 50 negative, 50 isolation/transaction and 30 limit-boundary cases. Declared capabilities qualified; offline packages on three hosts.
P2 Workflow beta	Incrementality; workers; watch/affected selection; editor/DAP; replay; selected async/Flow	Cold/warm, serial/parallel and IR/VM equivalence; fault injection; three project trials; measured budgets.
P3 Version 1.0	Stable protocols; mature common test surface; qualified legacy/mixed versions; release tooling	At least 2000 meaningful fixtures, five frozen corpora, no unexplained declared-scope mismatch, published compatibility/performance, all release gates.
P4 Expansion	Richer APIs, automation/sharing, search, locks and service adapters	Each addition individually promoted through evidence and benchmarks.

SHIP-01 P0 MUST include invalid source, wrong assertion, unsupported API, unqueried field, failed validation and state-leak controls. Freeze contracts only after the vertical slice exposes interactions. Arithmetic throughput alone cannot clear the gate.

SHIP-02 Before P1 scope freeze, list included API overloads and metadata forms with evidence owners. Core families are assertions, strings/numbers/dates, collections, JSON, Schema, Database, Limits, Test and mocked callouts; naming a family does not promise all overloads. Publish exclusions in alpha.

SHIP-03 Dependency order is language/values; rows/queries; DML/triggers/validation; isolation/security; VM equivalence; baseline/index/incremental optimization; editor/replay; async/Flow expansion. Parallel development is encouraged behind stable contracts; missing semantic dependencies cannot be bypassed for speed.

SHIP-04 Counts alone do not qualify releases. Fixtures must distinguish plausible defects. Every declared capability passes its matrix and unsupported cases block correctly. Keep counterexamples when reducing a claim rather than deleting failing evidence.

SHIP-05 v1 requires diagnostics/source maps, offline operation, reproducible builds, profiles, input handling, cancellation/recovery, selection accounting, contributor documentation and the performance budgets. The competitor-relative stretch target is separately reported; missing it cannot justify weakened accuracy or false claims.

SHIP-06 Estimate delivery after P0 measurements and a capability census of target corpora. Organize around compiler/runtime, data/security/automation, conformance and developer experience. More parallel implementers cannot compensate for unresolved shared semantics.

21 Risks and architectural responses

Risk	Response	Closing evidence
Incorrect local pass	Separate outcome, compatibility, assurance; block unknown effects	Negative controls and platform observations.
Version drift	Separate release/component policies and immutable profiles	Mixed-version/settings cases and reviewed revisions.

Risk	Response	Closing evidence
Numeric/query shortcuts	Explicit values/operators and scan reference	Precision, null, collation and plan comparisons.
Snapshot leakage	Immutable rows and fresh guest state	Shuffle, failure reuse, setup and worker equivalence.
Unsound affected selection	Conservative closure and audit mode	Mutated source/metadata compared with full runs.
Store becomes a database project	Isolated test transactions only	Baseline/overlay budgets without unnecessary durability.
API breadth delays delivery	Narrow releases driven by project capability census	Real-project completion and prioritized blockers.
Shared reference-engine bug	Independent platform evidence and mutations	Three-way comparison and reviewed expectations.
Great microbenchmark, poor workflow	Measure setup, edits, failures and reports	Complete-workflow distributions on fixed corpora.

22 Design checkpoint and reference policy

The committed direction is Rust, explicit Apex semantics, a qualified bytecode interpreter, native transactional test worlds, conservative incremental execution and open conformance. Earn speed by reusing immutable work, reducing allocations and scheduling isolated tests efficiently.

At P0, record decisions for parser revision, value representation, baseline format, profile catalogue, standard-object scope, hardware and corpora. Evidence may justify changes, but observable contracts must be preserved or explicitly revised.

Use primary platform documentation, independently collected observations and appropriately licensed public source. Documentation changes trigger profile review rather than silently changing existing behavior. Store provenance in the repository; shipped specifications and catalogues must stand on their own.

The product should be easy to trust, fast enough to stay in the edit loop, and open enough for users to diagnose and fix the next compatibility gap themselves.